a Date object. RSpec would attempt to use the date as a matcher object, and the spec would fail.

Next, the matches? method on line 14 looks like the DSL version, except that we need to hang on to the @account so that the failure messages can access it.

We have to define the description method explicitly on line 19, rather than having RSpec generate it from the names of the matchers and modifiers. The description_of helper (from the Composable module) is like calling inspect, but with special handling for matchers. We use it here since we're defining a higher-order matcher.

The remainder of the methods—the failure messages and private helpers—are basically the same as their DSL counterparts.

RSpec Integration

So far, we've just defined a Ruby class, HaveABalanceOf. To make this code available to RSpec as a matcher, we need to define a helper method, have_a_balance_of:

12-creating-custom-matchers/10/custom_matcher/spec/support/matchers.rb

```
module AccountMatchers
  def have_a_balance_of(amount)
    HaveABalanceOf.new(amount)
  end
end

RSpec.configure do |config|
  config.include AccountMatchers
end
```

Here, we've put the helper in a module and configured RSpec to include the module, making the method available to our examples.

The matcher class looks similar to its DSL equivalent, but takes up a lot more space and has to include quite a bit of boilerplate code. In most cases, you'll be better off using the DSL to define your matchers.

However, in certain situations the custom matcher class is a better fit:

- If your matcher is going to be used hundreds or thousands of times, writing your own class avoids a bit of extra overhead inherent in how the DSL is evaluated.
- Some teams prefer more explicit code.
- If you leave out the RSpec::Matchers::Composable mixin, your matcher won't have any dependencies on RSpec and will work in non-RSpec contexts.